FA-STAC: An Algorithmic Framework for Fast and Accurate Coupling Aware Static Timing Analysis

Debasish Das, *Student Member, IEEE*, Ahmed Shebaita, Hai Zhou, *Senior Member, IEEE*, Yehea Ismail, *Senior Member, IEEE*, and Kip Killpack

***Abstract*—This paper presents an algorithmic framework for fast and accurate static timing analysis considering coupling. With technology scaling to smaller dimensions, the impact of coupling induced delay variations can no longer be ignored. Timing analysis considering coupling is iterative, and can have considerably larger run-times than a single pass approach. We propose two different classes of coupling delay models: heuristic-based coupling model and current source-based coupling model, and present techniques to increase the convergence rate of timing analysis when such coupling models are employed. Our proposed coupling model show promising accuracy improvements compared to SPICE. Experimental results on ISCAS85 benchmarks validates the effectiveness of our efficient iteration scheme. Our iteration algorithm obtained speedups of up to 62.1% using a heuristic coupling model while 2.7× using a current-based coupling model in comparison to traditional approaches.**

***Index Terms*—Algorithm, crosstalk, mathematical modeling, signal integrity, timing analysis, timing verification.**

I. Introduction

WITH THE progress of deep sub-micrometer technologies, shrinking geometries have led to a reduction in the self-capacitance of wires. However, the aspect-ratio of modern interconnects is over 2.0 for intermediate wiring layers, and is expected to increase. This indicates that interconnect coupling capacitances have grown to dominate the total interconnect capacitance. Kumar [1] showed that in recent DSM technologies, coupling capacitances could be five times larger than the vertical capacitances. The total parasitic capacitance of an interconnect is given by

$$C_{eq} = C_g + \sum_j \mathrm{MCF}_j \times C_{cj} \tag{1}$$

Manuscript received May 30, 2008; revised January 20, 2009 and September 01, 2009. First published December 11, 2009; current version published February 24, 2011. This work was conducted at Northwestern University and supported by the National Science Foundation under CCR-0811270, and a grant from Intel. This work is an extended version of a paper from the Proceedings of ICCD, October 1–4, 2006, San Jose, CA.

D. Das, H. Zhou, and Y. Ismail are with the Department of Electrical Engineering and Computer Science, Northwestern University, Evanston, IL 60201 USA (e-mail: ddas@northwestern.edu; haizhou@northwestern.edu; ismail@eecs.northwestern.edu).

A. Shebaita is with the Information Technology Department, IBM, Valencia 46020, Spain (e-mail: a-shebaita@northwestern.edu).

K. Killpack is with Converged Core Development Organization, Intel Corporation, Hillsboro, OR 97124 USA (e-mail: kip.killpack@intel.com).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TVLSI.2009.2035323

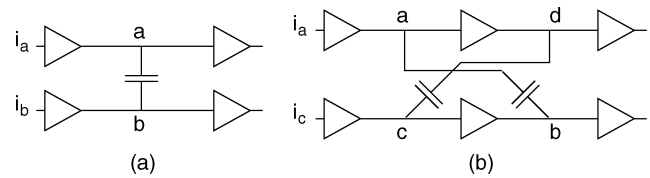

Fig. 1. Timing analysis with crosstalk is a mutual dependence problem: (a) local problem and (b) global problem.

where C_g is the interconnect's equivalent ground capacitance, C_{cj} is the interconnect's coupling capacitance with coupled neighbor j, and C_{eq} denotes the interconnect's equivalent total capacitance. MCF_j is some factor (often termed Miller coupling factor) that depends on the switching windows on the interconnect and j. Prior work has calculated the switching factors to be within (0, 2) for step transitions [2], within (−1,3) for ramp delay models [3], [4], and within (−1.885, 3.885) for exponential transitions [5].

With an increasing significance of coupling, ignoring the impact of coupling induced delay variations produces results far off from the reality. The simplistic model proposed in [2] is not accurate enough to capture the true impact of coupling on timing. It is well known that timing and coupling (or crosstalk) are mutually dependent. For example, consider the two coupled nets in Fig. 1(a). The switching time on net a is dependent on the switching time on net b. However, the switching time on net b is not fixed; it is dependent on the switching time on net a.

Present approaches to timing analysis considering crosstalk are iterative. Given input arrival times and starting with an initial assumption of crosstalk induced delays, the timing information on all interconnects are iteratively updated until convergence. For large designs, the run-times may be significant if the updates are not done efficiently. It is therefore critical to develop approaches to speeding up the analysis and improving the convergence rate.

Current approaches use either continuous coupling delay models [6] or discrete models (that employ Miller factors) [2], [4], [7]. In [6], three scheduling approaches, namely *dynamic event time*, *static event time*, and *smart global* are proposed, and the convergence time is compared. However, there is no thorough study on the convergence rate for different iterative schemes and the work in [6] leaves space for improvement.

Zhou [8] established a theoretical foundation for static timing analysis with crosstalk for both continuous and discrete models. A *chaotic iterative scheme* in this work provided the validity for exploring efficient iteration schedules. The approach in [6] initially assumes a situation of crosstalk delays (often the worst

1063-8210/$26.00 © 2009 IEEE
Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

case situation). Subsequently, timing information is updated iteratively until convergence. This approach does not exploit the circuit and coupling structures. We propose speeding-up techniques that can greatly improve the convergence rate of a static timing analyzer such as [6] that consider realistic coupling delay models.

In this paper, we present a fast and accurate static timing analyzer *FA-STAC* that has the following novel components:

1) waveform-based accurate coupling model;
2) efficient iteration mechanism.

We present two variants of waveform-based accurate coupling model. We call the first variant as heuristic-based coupling model while the second variant is referred to as current source-based coupling model. Heuristic-based coupling model is an extension of [3] to timing analysis and therefore we encourage readers to refer [3] for the accuracy of the heuristic. Based on our experience for process nodes smaller than 180 nm, such heuristic models suffer from significant inaccuracies. Due to their efficiency such models will be used for early stages of design to generate realistic bounds on the switching windows but we need a spice accurate model like the current source-based coupling model for sign-off stages of design.

Efficient iteration mechanism of *FA-STAC* decouples the iterative analysis algorithm from underlying coupling model. Coupling models can be selected and plugged into our iteration algorithm based on the accuracy versus performance tradeoff designer needs. Our iteration mechanism uses structural information of the coupled circuit to speed up the convergence rate of iterations during timing analysis. The iteration mechanism first partitions all couplings into global and local groups, and then uses different orderings to iterate through them. It turns out that as the coupling model complexity increases the performance overhead for structural analysis becomes insignificant as the overall performance of the algorithm. We compare the performance of our framework with the algorithm proposed in [6].

The rest of this paper is organized as follows. We present our heuristic-based coupling delay model in Section II and current source based coupling delay model in Section III. Section IV describes the proposed efficient iteration mechanism to increase the convergence rate of iterations. We present our experimental results in Section V. Conclusions and future work are presented in Section VI.

II. Heuristic-Based Coupling Model

Signal propagations in a circuit can be represented as a directed acyclic graph (DAG) while crosstalk couplings form bidirectional edges in the DAG. We have a general directed graph on the signals containing the circuit and the coupling structure. Formally, we define this general directed graph as $G = (V, E)$, where G is partitioned into two subgraphs $G^C = (V, C)$ and $G^F = (V, F)$. The edges in G^C are the bidirectional coupling edges and the edges in G^F are the fan-in edges. The gates of the combinational circuit are represented by the elements of set V. Graph G basically represents a timing graph in presence of coupling generated out of a given combinational circuit. We present following definitions for our derivations of the coupling model.

Definition 1: Rise delay window for a gate $i : i \in G$ is defined as an ordered pair (rd_i^l, rd_i^h), where rd_i^l represents the minimum possible rise delay, and rd_i^h represents the maximum possible rise delay at the output of gate i.

Definition 2: Fall delay window for a gate i is defined as an ordered pair (fd_i^l, fd_i^h), where fd_i^l represents the minimum possible fall delay, and fd_i^h represents the maximum possible fall delay at the output of gate i.

Clearly Definition 1–2 can be interpreted as all the possible points in time where the output at gate i can switch from one logic level to the other logic level. It also gives an intuition behind the importance of using rise and fall windows in coupling analysis since it determines the MCFs as shown in (1).

Definition 3: Rise slew window for a gate $i : i \in G$ is defined as an ordered pair (rs_i^l, rs_i^h), where rs_i^l represents the minimum possible rise slew, and rs_i^h represents the maximum possible rise slew at the output of gate i.

Definition 4: Fall slew window for a gate i is defined as an ordered pair (fs_i^l, fs_i^h), where fs_i^l represents the minimum possible fall slew, and fs_i^h represents the maximum possible rise slew at the output of gate i.

We also present two more definitions used in our derivations later.

Definition 5: A switching point is defined as a two-tuple (d, s), where d is an event from rise delay window or fall delay window and s is an event from rise slew window or fall slew window.

Definition 6: A switching window is defined as a rise or fall delay window with a fixed slew computed from the bounds of rise and fall slew windows.

Based on Definitions 1–4 of interacting gates, the coupling factors can vary significantly. We next present an example for illustration.

A. Motivational Example

We use an example as shown in Fig. 2 to motivate the need for accurate crosstalk delay variation estimation during timing analysis. Nets $I1$ and $I2$ fanin to pin 1 and pin 2 of gates $G1$ and $G2$. Rise and fall delay windows on net $I1$ are [2,4] and [2.5,3.5]. Similarly on net $I2$ the windows are [3,5] and [3.5,4.5]. Assuming $G1$ and $G2$ are identical NAND gates, timing arcs *arc1* and *arc2* are identical. Each arc contains a four-tuple timing information, which corresponds to rise delay, rise transition, fall delay, and fall transition, respectively. Also let us assume that net $I1$ and $I2$ have similar rise and fall slew windows given by [0.2,0.6] and [0.4,0.8]. In Fig. 2, we have not shown the ground capacitance. We denote the ground capacitances corresponding to a gate i as C_g^i (it is the lumped capacitance corresponding to net O^i).

For the sake of simplicity, assume that we have a single rise slew $(0.2+0.6)/2$ and fall slew $(0.4+0.8)/2$. Consider *arc1* as positive unate and *arc2* as negative unate. Let the rise delay on *arc1* due to input rise slew as 0.4 and loading cap of CG^1+C_c be 0.6. Input window [2,4] transforms to [2.6,4.6]. Let the fall delay on *arc2* due to input fall slew as 0.6 and loading cap of $C_g^1 + C_c$ be 0.8. Input window [3.5,4.5] transforms to [4.3,5.3]. The final output rise window on gate $G1$ is given by [2.6,5.3]. Also the window of rise slew created on output of $G1$ due to input rise slew of 0.4 and input fall slew of 0.6 be [0.5,0.7]. Similarly the final output fall window on gate $G1$ comes from the windows

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

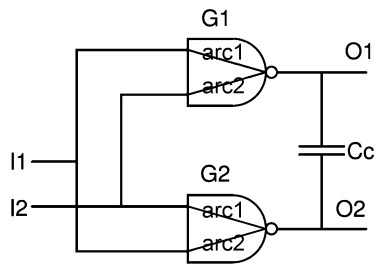

Fig. 2. Accurate analysis of coupling capacitance.

[2.5 + 0.8,3.5 + 0.8] and [3 + 0.6,5 + 0.6] which is [2.5,5.6]. The fall slew window will be same as [0.5,0.7] due to symmetry. Note that $G1$ and $G2$ are similar gates. The difference lies in the fact that $G2$ has a loading capacitance of $C_g^2 + C_c$. Suppose the difference in capacitances add 0.4 to all delay and 0.1 to all slew calculations on *arc1* and *arc2*. Thus at $G2$ we get the output rise delay window as [3.0,5.8] and fall delay window as [2.9,6.0]. Also the output rise and fall slew windows are given by [0.6,0.8]. Note that we have already simplified our calculations by considering a single slew. If I1 and I2 are nets connected to primary inputs, this assumption is valid. But for a general circuit, both the minimum and maximum slew values must be considered to get accurate output switching windows.

For ramp-based model, as shown in [3], a switching point at output of $G1$ and a switching point from $G2$ can give rise to coupling capacitance as large as $3 \times C_c$.

B. Coupling Factor Computation

Since static timing analysis is corner case analysis, we can always take the maximum and minimum coupling factors to calculate the effect of coupling capacitances on fall and rise delay window. Given a coupling edge $e = (v, a)$ with intrinsic coupling capacitance as C_c^e, where v and a has rise/fall delay and slew windows as defined earlier. We are seeking to find best and worst case coupling factors that might come from this configuration and thus determine the effective coupling capacitance between the nodes v and a. Note that rise and fall delay windows basically represent the maximum and minimum arrival times after which the gate output starts to change from its steady state. Therefore both the arrival times and the slews are extremely important to determine the coupling factors as it captures the shape of the waveforms accurately. According to the convention of [3] we assume one of the nodes v as *victim* while the other one a is considered as *aggressor*, though considering one particular node as the victim, while the other as an aggressor is artificial for timing analysis. The idea is that we consider one node as an victim and the rest of the nodes connected to it by coupling edges as aggressor, and the timing analysis procedure takes care of the fact that the victim node will also be considered as an aggressor later.

Definition 7: Overlap ratio k is defined as the ratio of the aggressor's output waveform that overlap with that of victim threshold voltage.

Based on the definition we have five possible Equations for k. The first four conditions are as shown in Fig. 4(a)–(d). The last equation takes care of the case when there is no overlap between the victim and aggressor waveforms. We multiply the intrinsic

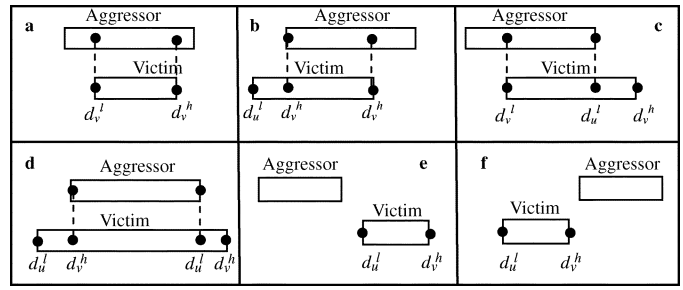

Fig. 3. Window overlap cases.

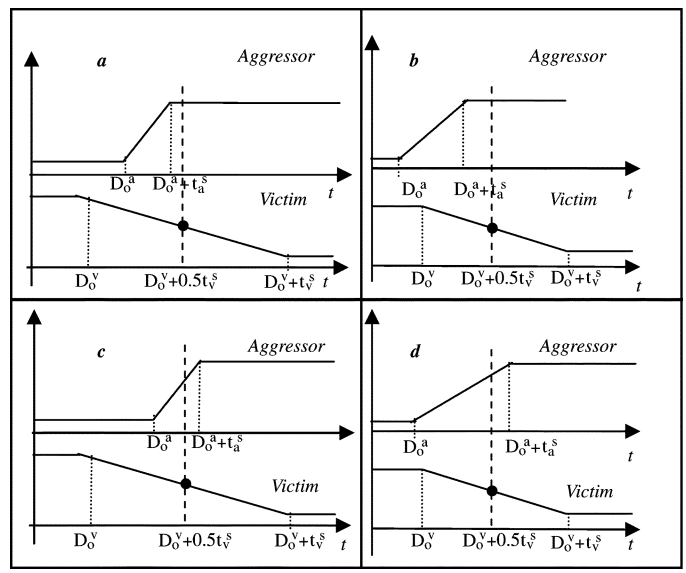

Fig. 4. Coupling factor computation.

coupling capacitance C_c^e with k to get the effective coupling capacitance

$$k = \begin{cases} 1, & \text{Figure 4(a)} \\ \frac{D_o^a + t_a^s - D_o^v}{t_a^s}, & \text{Figure 4(b)} \\ \frac{D_o^v + 0.5t_v^s - D_o^a}{t_a^s}, & \text{Figure 4(c)} \\ \frac{0.5t_v^s}{t_a^s}, & \text{Figure 4(d)} \\ 0, & \text{No overlap condition.} \end{cases} \quad (2)$$

Now we need to choose the points D, t^s from the delay and slew windows that gives the worst and best possible coupling factors for rise and fall delay computation. Note that worst case coupling factors come from rise and fall or fall and rise delay windows on victim and aggressor while best case coupling factors come from rise and rise or fall and fall delay windows, respectively. After we choose the points D_o^a, D_o^v, t_a^s, and t_v^s according to Fig. 5, we consult Fig. 4 for the corresponding waveforms that the chosen points from the windows will generate. Accordingly we obtain k from the equations given above. Coupling parameter computation as shown in Fig. 5 uses different conditions of switching window overlap as shown in Fig. 3.

C. Directed Search Technique

Section II-B presented a simple approach to compute coupling factors using static timing analysis conventions. Such a technique is quite efficient but it ignores the correlation between

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

Figure 3(a)
Max
$D_o^a = d_v^h, t_a^s = s_u^l$ AND $D_o^v = d_v^h, t_v^s = s_v^h$
Min
$D_o^a = d_v^l, t_a^s = s_u^l$ AND $D_o^v = d_v^l, t_v^s = s_v^h$
Figure 3(b)
Max
$D_o^a = d_v^h, t_a^s = s_u^l$ AND $D_o^v = d_v^h, t_v^s = s_v^h$
Min
$D_o^a = d_u^l, t_a^s = s_u^l$ AND $D_o^v = \mathrm{best}(d_u^l, d_v^l), t_v^s = s_v^h$
Figure 3(c)
Max
$D_o^a = d_u^h, t_a^s = \mathrm{worst}(s_u^l, s_u^h)$ AND
$D_o^v = \mathrm{worst}(d_u^h, d_v^h), t_v^s = s_v^h$
Min
$D_o^a = d_v^l, t_a^s = s_u^l$ AND $D_o^v = d_v^l, t_v^s = s_v^h$
Figure 3(d)
Max
$D_o^a = d_u^h, t_a^s = \mathrm{worst}(s_u^l, s_u^h)$ AND
$D_o^v = \mathrm{worst}(d_u^h, d_v^h), t_v^s = s_v^h$
Min
$D_o^a = d_u^l, t_a^s = s_u^l$ AND $D_o^v = \mathrm{best}(d_u^l, d_v^l), t_v^s = s_v^h$
Figure 3(e)
Max
$D_o^a = d_u^h, t_a^s = \mathrm{worst}(s_u^l, s_u^h)$ AND
$D_o^v = \mathrm{worst}(d_v^h, d_v^l), t_v^s = s_v^h$
Min
$D_o^a = d_u^h, t_a^s = s_u^h$ AND $D_o^v = d_u^l, t_v^s = s_v^h$
Figure 3(f)
Max
$D_o^a = d_u^l, t_a^s = s_u^l$ AND $D_o^v = d_v^h, t_v^s = s_v^h$
Min
$D_o^a = d_u^l, t_a^s = s_u^l$ AND $D_o^v = \mathrm{best}(d_v^l, d_v^h), t_v^s = s_v^h$

Fig. 5. Parameter selection for coupling factor computation.

input timing event and worst MCF. An example will further clarify the idea. Suppose the input rise delay window is given by [2.0,4.0] and for max delay analysis we choose the highest slew from the window; 0.6. We denote a timing event as a ordered set of delay and associated slew. Suppose there are two timing events $T1 = (3.9, 0.6)$ and $T2 = (4.0, 0.6)$. $T1$ resulted in MCF of 2.5 while $T2$ gave MCF of 2.4. But delay push-out due to $T1$ can still be less than the delay push-out due to $T2$ and thus the event $T2$ still provides the worst case MCF of 2.4. Delay push-outs corresponding to each timing event should be taken into account for more accurate coupling analysis.

Based on the discussion above, for worst MCF computation, searching the space spanned by input delay and slew windows are necessary. Our directed search approach is based on this intuitive idea and we present an efficient approach to search for the input timing event which results in the worst delay push-out rather than finding a worst coupling factor. Fig. 6 shows the relations that govern the MCF for both the victim and aggressor in each of the four cases shown in Fig. 4. Fig. 6 shows the MCF of both victim and aggressor for opposite and similar switching

	Victim MCF	Aggressor MCF
a	1±2k	$1 \pm \frac{t_a^s}{t_v^s}$
b	1±2k	$1 \pm \frac{t_a^s}{t_v^s}(2k-1)$
c	1±2k	$1 \pm \frac{t_a^s}{t_v^s}$
d	1±2k	$1 \pm \frac{0.5}{k}$

Fig. 6. Victim and aggressor MCF for various coupling scenarios.

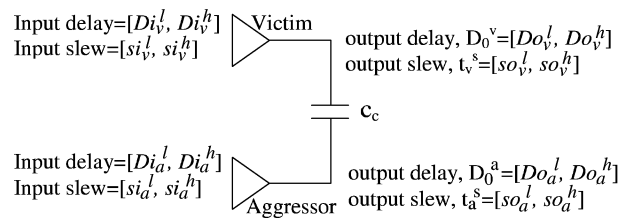

Fig. 7. Victim and aggressor equivalent circuit.

cases such that k is the ratio of aggressor waveform that overlap with that of the victim up till the victim's threshold voltage v_{th}, t_a^s is the aggressor output slew, and t_v^s is the victim output slew. The "+" operator corresponds to the opposite switching case while the "−" operator corresponds to the similar switching case.

k is the overlap ratio as shown in Definition 7. Strict lower and upper bounds of 0 and 1 holds for k. Bounds of k plays an important role in our technique as mentioned in Section II-C1.

Two windows are to be determined at the output of each gate. The first window is the arrival delay window, $[D_0^l, D_0^h]$, defined as the possible output delay values after which the gate output starts to change from its steady state. The second window is the slew window, $[s_0^l, s_0^h]$, defined as the window of the possible slews at the output of the gate. These two windows are computed based on the given delay and slew windows at the input of both the victim and aggressor. Fig. 7 shows a circuit diagram for a victim, aggressor, the coupling capacitance between them, and the slew and delay intervals.

Fig. 8 shows the equivalent decoupled circuit of that shown in Fig. 7. The output delay at the victim and aggressor gates is given by

$$D_o^v = D_i^v + t_v^d \tag{3}$$
$$D_o^a = D_i^a + t_a^d \tag{4}$$

where $D_i^v \in [Di_v^l, Di_v^h]$, t_v^d is the delay through the victim gate, $D_i^a \in [Di_a^l, Di_a^h]$, and t_a^d is the delay through the aggressor gate.

A cell library is used that has slew/delay tables associated with each gate for both rising and falling output waveform. These tables are looked up with the input slew and the output effective capacitance and give both the corresponding output delay and the output slew. It was observed that the output delay and slew of this cell library is linear with the input slew and load

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

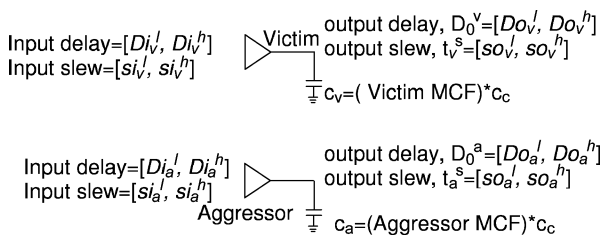

Fig. 8. Victim and aggressor decoupled circuit.

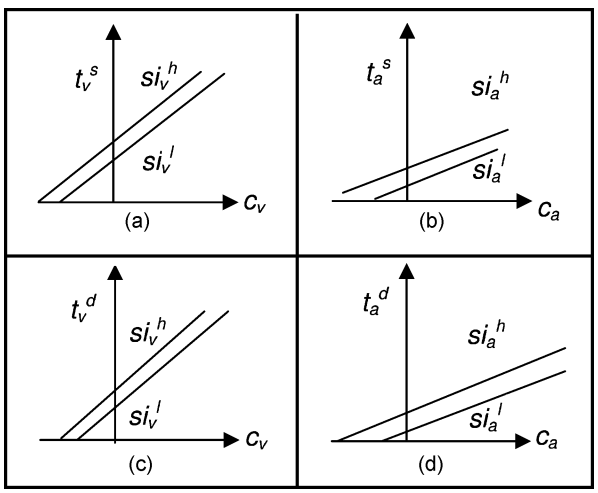

Fig. 9. Plot of aggressor and victim output delays and slews.

capacitance. This observation can be explained by assuming the α power law delay and slew approximations.

The output delay t^d and the output slew t^s as functions of the output capacitance and input slew are given by [15]

$$t^d = \left(\frac{1}{2} - \frac{1 - \frac{v_{\text{th}}}{V_{\text{DD}}}}{1+\alpha}\right) t^s + \frac{C_L V_{\text{DD}}}{2 I_o} \tag{5}$$

$$t^s = \frac{C_L V_{\text{DD}}}{I_o}\left(\frac{9}{8} + \frac{V_{D_o}}{0.8 V_{\text{DD}}} \ln\left(\frac{10 V_{D_o}}{e V_{\text{DD}}}\right)\right) \tag{6}$$

where C_L is the output capacitance, α is the velocity saturation index, V_{D_o} is the drain saturation voltage at $V_{GS} = V_{\text{DD}}$, I_o is the drain saturation current at $V_{GS} = V_{\text{DD}}$. It is clear from (5) and (6) that the output delay and the output slew are linearly related to the output capacitance and the input slew. Thus, the library tables for both the victim and aggressor gates can be visualized as shown in Fig. 9.

Fig. 9 shows the linear relation between the output delay and slew and the output capacitance for a given input slew range $[s_i^l,\ s_i^h]$. To determine worst/best delay and slew each of the four possible cases shown in Fig. 4 should be considered. The worst/best case delay computed based on each case should be compared with those computed from the other cases to find the ultimate worst/best delay and slew. The following section describes the details of obtaining the worst case delay assuming the scenario shown in Fig. 4(a).

1) Worst Case Delay Computation: The worst case computation involves spanning the possible values of k that ranges from 0 to 1. For each value of k the victim capacitance is computed as

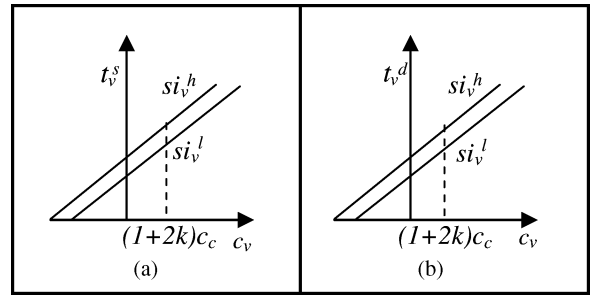

Fig. 10. Victim delay and slew values at $(1+2K)C_c$.

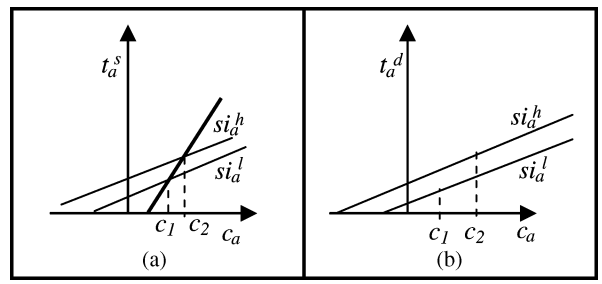

Fig. 11. Graphical solution for aggressor slew and delay.

$(1+2k)C_c$ as shown in Fig. 6. This value of the victim capacitance results in a set of values for the victim delay and slew as shown in Fig. 10. As shown in Fig. 6, the aggressor capacitance for this case(a) is

$$c_a = \left(1 + \frac{t_a^s}{t_v^s}\right) c_c. \tag{7}$$

Thus, the equation that gives the aggressor slew in terms of the aggressor effective capacitance c_a is given by (8)

$$t_a^s = t_v^s \left(\frac{c_a - c_c}{c_c}\right) \tag{8}$$

where t_v^s is obtained as shown in Fig. 10(a). Fig. 11(a) shows the plot of Fig. 10 on top of Fig. 9(b).

The intersection point between Figs. 10 and 9(b) give the aggressor capacitors $[c_1, c_2]$. Each aggressor delay points t_a^d can be obtained by substituting with one of the aggressor capacitors $[c_1, c_2]$ as shown in Fig. 11(b). Each set of the values obtained t_v^s, t_v^d, t_a^s, t_a^d should be tested to see whether it satisfies the overlap scenario shown in Fig. 4(a). The sets that satisfy the scenario are then compared and the worst delay and slew that can occur in this scenario is determined. The previous computation steps should be repeated with all the possible scenarios shown in Fig. 4(a)–(d) and the worst out of all the scenarios is the absolute worst case delay.

2) Convergence: Directed search for 2 coupled nets is presented in Section II-C. But in a general circuit a net is coupled with more than one nets. As shown in Fig. 12, net O1 is coupled with net O2 with coupling capacitance C_c and net O3 with coupling capacitance C13, net O2 is coupled with O1 and O3 while net O3 is coupled with O1 and O2. Application of directed search on a cluster of more than one coupled nodes is of practical interest. Each vertex of the timing graph G has rise delay window (rd_i^l, rd_i^h), fall delay window (fd_i^l, fd_i^h), rise

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

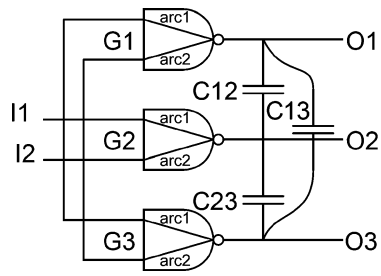

Fig. 12. Fixed point iteration in directed search.

slew window (rs_i^l, rs_i^h), and fall slew window (fs_i^l, fs_i^h), respectively.

Definition 8: k-cluster is defined as the k-tuple $\langle V, A^0, \ldots, A^{k-2}\rangle$, where

$$V, A^0, A^1, \ldots, A^{k-2} \in V \tag{9}$$

$$\forall i \in (0, \ldots, k-2) : (V, A^i) \in C \tag{10}$$

where V is called as the victim node while A^i are its aggressors.

Based on Definition 8, we formulate our problem

Problem 1 (Directed Search on k-Cluster): Given a k-cluster with initial 1C switching windows, find the best and worst case coupling capacitance that appears on victim V due to aggressors $A^i : i \in (0, \ldots, k-2)$.

Given a k-cluster $\langle V, A^0, \ldots, A^{k-2}\rangle$, a set of switching points from appropriate windows of $V, A^0, \ldots, A^{k-2}$ will give rise to best and worst case coupling capacitances. When one is seeking for best case capacitance, rise/rise and fall/fall windows should be considered. For worst case coupling capacitance, rise/fall window of V and fall/rise window of $A^0, A^1, \ldots, A^{k-2}$ should be considered. We consider maximum victim slew for worst case analysis. Minimum victim slew is considered for the best case. For the rest of the discussion we will consider worst case analysis as best case is symmetric to it. The following set gives all possible switching points in a k-cluster:

$$\left\{X^V, X^{A^0}, X^{A^1}, \ldots, X^{A^{k-2}}\right\}. \tag{11}$$

Here X^j is the set of all switching point at node j, where $j \in (V, A^0, A^1, \ldots, A^{k-2})$.

Theorem 1: We find points $x \in X^V$ and $x^i \in X^{A^i}$, where $i \in (0, 1, \ldots, k-2)$ and form a set

$$\left\{(x, x^0), (x, x^1), \ldots, (x, x^{k-2})\right\}. \tag{12}$$

Equation (13) is a permutation of (12)

$$\left\{(x, \hat{x^0}), (x, \hat{x^1}), \ldots, (x, \hat{x^{k-2}})\right\}. \tag{13}$$

Equation (13) forms a totally ordered set.

Proof: We define a relation k-inclusion (that is $\sqsupseteq$)

$$k^{(x,x^i)} \geq k^{(x,x^j)} \Rightarrow (x, x^i) \sqsupseteq (x, x^j) \tag{14}$$

$$k^{(x,x^i)} < k^{(x,x^j)} \Rightarrow (x, x^j) \sqsupseteq (x, x^i) \tag{15}$$

where $k^{(x,x^i)}$ is the overlap ratio between points x and x^i. By the definition of $\sqsupseteq$ and by the fact that a unique k exists between 0 and 1 for all two-tuples (x, x^i), we can arrange (12) in terms of increasing k. Equation (13) formed by the permutation of (12) is a totally ordered set. ■

According to lattice theory [16] a partially ordered set (which holds for a total order as well) forms a complete lattice if any subset has a least upper bound $(\top)$ and greatest lower bound $(\bot)$ for its elements. Our set has a greatest lower bound when k is 0 and a least upper bound when k is 1 for all of its elements. Thus the set forms a complete lattice and if there is a order preserving transformation T [8], we can find the fix-points of this set using chaotic iteration scheme.

Theorem 2: Directed Search between a victim and an aggressor is an order preserving transformation. We call the transformation as DS

$$\forall i : (x, x^i) \sqsupseteq (\tilde{x}, x^i) \Rightarrow DS(x, x^i) \sqsupseteq DS(\tilde{x}, x^i). \tag{16}$$

Proof: Complete description of directed search is given in Section II-C. Directed search between a victim and an aggressor come up with the worst possible coupling effect on the victim due to the aggressor and that changes the victim waveform by increasing the overlap ratio. Therefore

$$(x, x^i) \sqsupseteq (\tilde{x}, x^i) \Rightarrow k^{(x,x^i)} \geq k^{(\tilde{x},x^i)}. \tag{17}$$

By property of DS

$$DS(x, x^i) = (\hat{x}, x^i) \text{ where } k^{(\hat{x},x^i)} \geq k^{(x,x^i)} \tag{18}$$

$$DS(\tilde{x}, x^i) = (\bar{x}, x^i) \text{ where } k^{(\bar{x},x^i)} \geq k^{(\tilde{x},x^i)}. \tag{19}$$

From (17)–(19)

$$k^{(\hat{x},x^i)} \geq k^{(\tilde{x},x^i)} \Rightarrow DS(x, x^i) \sqsupseteq DS(\tilde{x}, x^i).$$

■

We present a recursive formulation to apply the transformation DS on the set given by (13)

$$\begin{aligned}
&\bar{DS}\left\{(x, x^0), (x, x^1), \ldots, (x, x^{k-2})\right\} \\
&\quad = \bar{DS}\left(\bar{DS}\left\{(x, x^0), (x, x^1), \ldots, (x, x^{k-3})\right\}, \right. \\
&\qquad\qquad \left. \bar{DS}(x, x^{k-2})\right) \\
&\bar{DS}(x, x^i) \\
&\quad = DS(\hat{x}, x^i).
\end{aligned}$$

We always generate subsets of (13) during the recursive application of DS. Chaotic iterations will reach to an optimal fixed point when

$$\begin{aligned}
&\bar{DS}\left(\left\{(\bar{x}, x^0), (\bar{x}, x^1), \ldots, (\bar{x}, x^{k-2})\right\}\right) \\
&\qquad = \left\{(\bar{x}, x^0), (\bar{x}, x^1), \ldots, (\bar{x}, x^{k-2})\right\}.
\end{aligned} \tag{20}$$

When the fixed point iteration converge, we get an effective capacitance which corresponds to the worst effect on victim due to all its aggressors. A pseudocode of chaotic iteration scheme for worst case coupling capacitance calculation is shown in Fig. 13. The algorithm presented in Fig. 13 comes up with the worst case coupling capacitance on victim V due to all its aggressors. We call this capacitance as $C_c^{k-\text{cluster}}$. Similar formulation can be used to get the best case coupling capacitance.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

Algorithm: Worst-Coupling-Solver

```
Input: k-cluster < V, A^0, ..., A^{k-2} >
Output: Worst case coupling effect on V
begin
  C_c^{k-cluster} ← Σ_{i=0}^{k-2} C_c^i
  C_c^{iter} ← 0
  Node-Queue← A^0, A^1, ..., A^{k-2}
  while(Node-Queue is not empty)
    C_c^{iter} ← C_c^{k-cluster}
    a ← Pop(Node-Queue)
    Update C_c^{k-cluster} due to DS(V, a)
    if(C_c^{k-cluster} ≠ C_c^{iter})
       Push a into Node-Queue
  endwhile
end
```

Fig. 13. Worst case victim coupling capacitance computation.

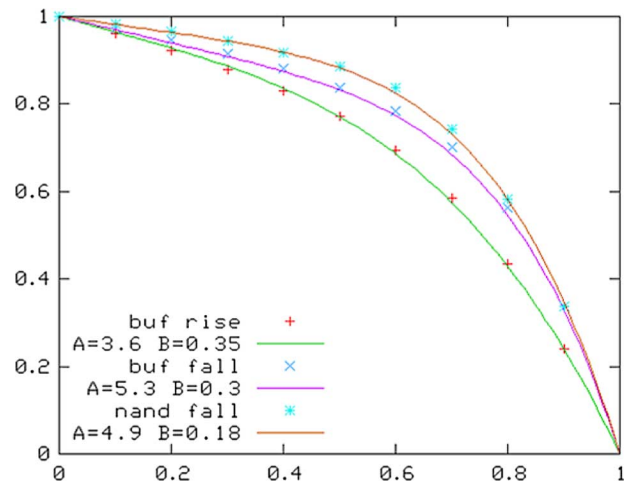

Fig. 14. $I_{\rm on}(V_o)/I_{\rm sat}$ compared to SPICE data.

III. Current Source-Based Coupling Model

Due to prominent nonlinearity of driver cells for process nodes below 90 nm, switch level static timing analysis techniques using simple resistive models [19] are no longer applicable. To address this issue, several researchers proposed current source models (CSMs) which model the gate nonlinear behavior with voltage controled dc current source and/or parasitic behavior with capacitances [20]–[23]. But existing current source models are not compatible with the delay and slew tables based library models such as nonlinear delay model (NLDM) and need Spice-based cell precharacterization. Also they are order of magnitudes slower than the effective capacitance-based resistive models [19]. Due to the performance issues at early stages of design using a CSM may be prohibitive and more efficient models are required. Imodel, our current-based logic cell model is developed to resolve the above shortcomings. Imodel, does not need any characterization and can extract the parameters from NLDM library. This advantage does not come with a big sacrifice and our experiments show crosstalk noise analysis with accuracy close to Spice. Details of the model and its application to timing analysis are presented in [18]. We briefly present the accuracy comparison of our model with SPICE.

A. Description of Imodel

First, we need a model of current versus output voltage for a cell that has fully turned on. Three-parameter are used for this. For output rising, and voltage normalized to $V_{\rm dd}$, the model is

$$I_{\rm on}(V) = I_{\rm sat}\left(1 - B \times V - (1-B) \times V^A\right)$$

with $0 < B < 1$ and $2 < A < 6$. Fig. 14 shows this curve compared to spice iv data for a buffer, rising (spice: +, model: green) and falling (spice: x, model: pink), and a two-input NAND gate, falling (spice: *, model: brown). Analytical model equations used in Fig. 14 are shown as follows:

$$\text{BUFFER RISE}: 1 - 0.35 \times x - 0.65 \times x^{3.6}$$
$$\text{BUFFER FALL}: 1 - 0.3 \times x - 0.7 \times x^{5.3}$$
$$\text{NAND FALL}: 1 - 0.18 \times x - 0.82 \times x^{4.9}.$$

This shows that this three-parameter formula is capable of modeling the shape of real devices.

The relevant NLDM data is delay and output slew at smallest input slew and at large capacitance, so that the output transition is slow enough that the device can be assumed to be fully turned on. The key shape measurement extracted from this data is the ratio of the derivative of delay with respect to output load and derivative of slew with respect to output load. It corresponds to the ratio of the integrals of 1/Ion(v) over the regions [0,0.5] and the [lo,hi] slew thresholds. But this is only one data point for the two parameters, A and B. In practice, if we have no other data, we set $B = 0.2$ and use this ratio to determine A. We extract an output capacitance C_o by expecting that at large output load C and short input slew we should have output slew proportional to $C + C_o$. The largest two output load points in the output slew table are used for this. A parameter S is defined as switching time. The model becomes

$$I(v,t) = \begin{cases} I_{\rm on}(V + 1 - t/S), & \text{if } 0 \le t \le S \\ I_{\rm on}(V), & \text{if } t > S. \end{cases} \tag{21}$$

When the model is driving very small capacitance, V will be approximately t/S. Accordingly, S is chosen so that the model matches output slew at low output load. Since the most effective aggressor alignment is often such that the aggressor is switching before the victim has begun to switch, and the victim's strength in this precharge region is determined by the opposite n/p type of transistor as determines the fully turned on current, we extract a conductance $G_{\rm hold}$ from the NLDM table with opposite rise/fall from the transition being modelled. The precharge current is

$$I(v,t) = -G_{\rm hold} \times V \text{ if } t < 0. \tag{22}$$

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

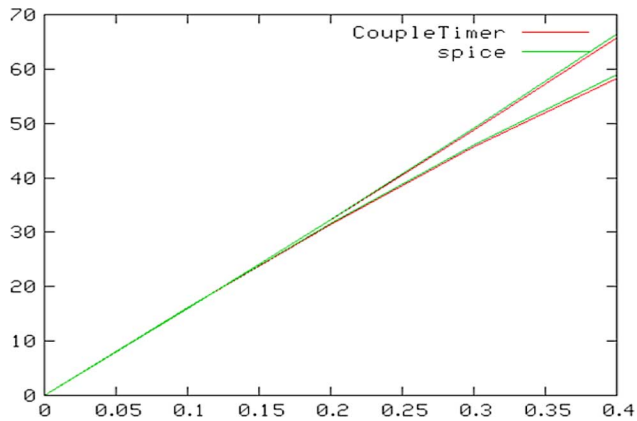

Fig. 15. Max delta delay versus $Cx/Ctot$.

We now have a six-parameter model where the parameters are I_{sat}, A, B, C_o, S, and G_{hold}. However, this model is not too strong in the switching region, as if the competing n and p channels were fully on. So we multiply $I(v,t)$ by a suppression factor $f(t) \leq 1$ in a region containing S. A default choice without introducing new parameters is a piecewise linear through the points $f(-S) = 1$, $f(0) = 0.2$, $f(S) = 0.2$, $f(2S) = 1$. This completes the description of the six-parameter model that can be extracted from NLDM data. We did not add miller capacitance because we could not extract it effectively from NLDM data. Given the richer CCS and CCS-noise libraries, miller capacitance should be added, and the turn on suppression factor $f(t)$ could be given additional parameters.

B. Accuracy Comparisons

We focus on the case of a single aggressor. Both the victim and aggressor drivers are modeled as above. Aggressor alignment is optimized within the timing window. In the results presented here, there is no wire resistance, and the circuit consists only of the two drivers, two grounded caps, and the coupling cap. Spice-like simulation is done with the $I(v,t)$ models, using the trapezoidal method and varying timesteps. The power V^A is approximated by a polynomial for speed. For Fig. 15 C_x represents the coupling capacitance while C_{tot} is the sum of ground and coupling capacitance of the victim. Also $X1$ and $X8$, respectively, represent smallest and largest size from cell library. Red lines in Figs. 16–19 indicates our crosstalk aware static timer's output (referred as CoupleTimer in Figs. 16–19) while the green line indicates SPICE simulation result.

1) *Crosstalk Delay Versus C_x/C_{tot}.*
In Fig. 15, we show the peak delta delay, for worst aggressor alignment in an infinite window, for an $X1$ buffer as victim, $X8$ buffer as aggressor, total victim cap $= 40$ ff, total aggressor cap $= 100$ ff, with varying coupling cap. The x-axis is the ratio of coupling capacitance to total victim capacitance.

2) *Crosstalk Delay Versus Alignment Curves.*
In Figs. 16–19, we show crosstalk delta delay versus aggressor alignment, compared to SPICE. Fig. 16 is late/slowdown delta delay for $\text{victim} = \text{BUFX1}$, $\text{aggressor} = \text{BUFX1}$, Fig. 17 is for $\text{victim} = \text{BUFX1}$, $\text{aggressor} = \text{BUFX8}$. Fig. 18 is for $\text{victime} = \text{aggressor} = 2 - \text{input}$ OR, $X8$. Fig. 19 is early/speedup delta delay for $\text{victim} = \text{NAND}X1$, $\text{aggressor} = \text{NAND}X8$.

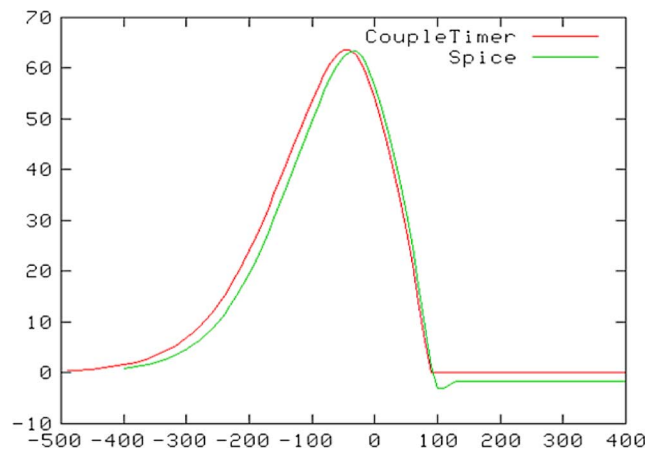

Fig. 16. Delta delay versus aggressor alignment.

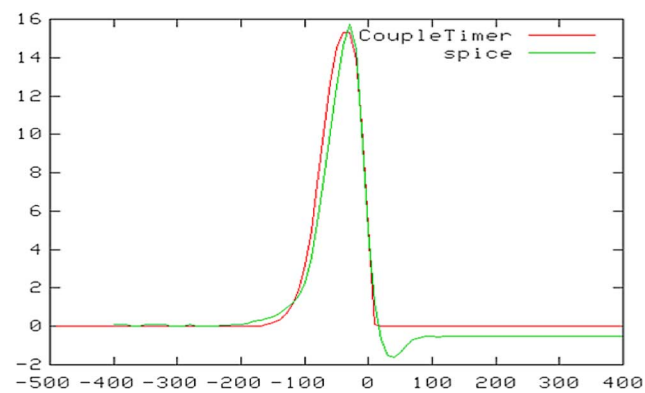

Fig. 17. Delta delay versus aggressor alignment.

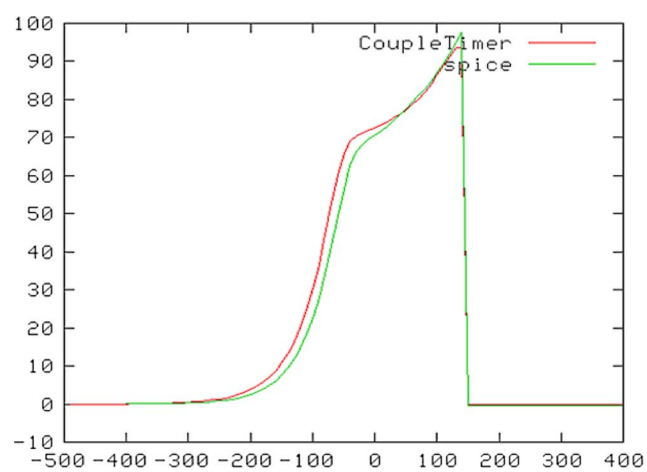

Fig. 18. Delta delay versus aggressor alignment.

Such a complex coupling model can be easily plugged into our efficient iteration mechanism to improve the accuracy of timing analysis.

IV. Efficient Iteration Mechanism

Static timing analysis considering coupling is an iterative approach. The efficiency of the current iterative approaches can

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

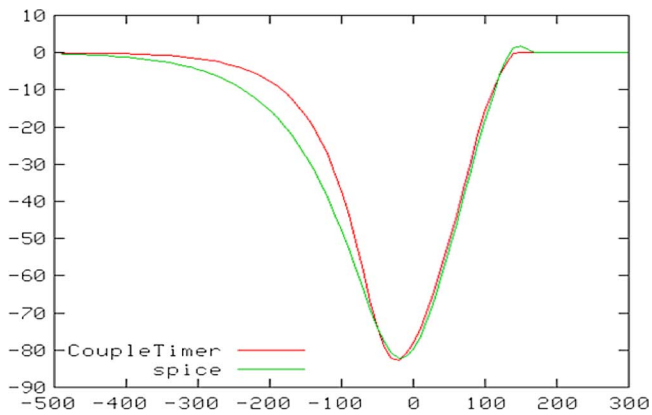

Fig. 19. Early delta delay versus aggressor alignment.

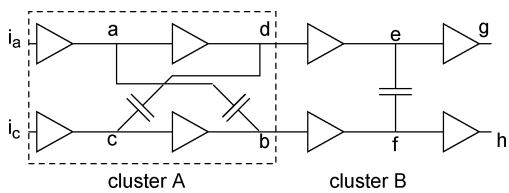

Fig. 20. Exploit circuit structure by clustering.

increase greatly if the number of iterations are reduced. Commonly, timing analysis is performed in a topological order of the circuit. As shown in Fig. 20, this approach updates the timing information in a way such that any update at d will be propagated to e, f, g, and h. If the update at d is not permanent, those propagations will be overwritten later, and previous calculations are thus wasted.

Zhou [8] revealed that as long as we are using a monotonic coupling delay model, the process will always converge to the same fixpoint no matter what iterative order is adopted (termed *chaotic iteration* [9]). This result forms the basis for finding an efficient ordering scheme to do the iteration for fast convergence.

One of the most useful properties of any iterative coupling analysis algorithm is that the coupling model should be decoupled with the analysis algorithm. Our iteration mechanism has the desired property. We discuss later in this section that our mechanism is amenable to adding an accurate model presented in Section III.

A. Clustering

Treating coupling edges as bidirectional edges we can identify strongly connected components in the graph with both circuit fan-out edges and coupling edges as clusters. Processing each cluster till convergence in the topological order among clusters will trim off the wasted calculations. As shown in Fig. 20 clusters A and B have different structures. B is simpler as any change in the window at e will only affect the windows at f through the coupling. We call it a *local cluster*. However cluster A has a different structure. Any change in window at a will affect the window at b due to coupling and window at d through fan-out. Now since the change in window at d can affect window at c which in turn can affect a gives rise to a feedback mechanism in A. In essence cluster A includes two local clusters interacting with each other. We call such a cluster as *global cluster*. Solving the timing analysis with crosstalk is then reduced to finding a convergent solution on a set of local and global clusters.

B. Coupling Edge as Feedback

If we always compute local clusters together [7], [10], [11], then we need to select a set of gate inputs as feedback edges, whose removal makes the structure acyclic. We do timing analysis in the acyclic part and if it changes the values on feedback edges, we start the next iteration. Given a general graph G corresponding to a combinational circuit which includes the coupling and fan–out edges, finding the feedback edges helps in reducing the number of iterations to reach the stable values of switching windows. Although there is no direct relation between the number of iterations and number of feedback edges, fewer feedback edges will give fewer possible value changes. However finding the smallest number of feedbacks is NP-hard on a general graph [12].

A good heuristic which we used to identify the feedback edges is to use the coupling edges as the feedback. This is based on the fact that if all the feedbacks are gate inputs, any changed values on them always need to be propagated. Studying the interactions in our system, we find that our system is a heterogeneous system. That is, there are two kinds of interaction relations: fan-in relation is simple but strong; coupling relation is complex but weak. Fan-in relation is simple because it is unidirectional, and it is strong because timing value on an input always influences timing value on an output. On the other hand, coupling relation is complex because it is bidirectional, and it is weak because timing value on one net may not always influence the value on the other (if they will not switch at the same time).

We can identify all feedback edges as the *Global* coupling edges. That is because the change of window on feedback edges will result in beginning the iteration again. Once we can identify all feedback edges, i.e., the edges whose nodes have a fan-in relation between them, we can delete all such edges. Global coupling edge Identification begins with considering all the coupling edges C potential candidates for global couplings. The algorithm selects each edge (u, v) out of the set C and see whether there is a path from vertex u to vertex v. A breadth-first-search [13] is started from the vertex u and we insert the edge (u, v) into the global cluster if vertex v is reached through the search. We call the coupling edges identified by the BFS-based algorithm as *primary* edges of C^G. We call the set of *primary* edges as C_P^G. C_P^G represents all feedback edges as discussed earlier.

C. Coupling Weight Assignment

We assign an weight to each edge $e \in C - C_P^G$, where weight represents how closely the nodes forming coupling edge e are coupled with each other. To determine the weight, we calculate the initial rise and fall delay windows on each node of the graph G. For the initial switching window computation we consider the capacitance as the sum of gate capacitance and the intrinsic coupling capacitance (MCF as 1). Based on the initial switching windows we calculate the overlap ratio k as described in Definition 7 for each coupling edge $e \in C - C_P^G$ and k is assigned as the weight to each edge e.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

Definition 7 is an indicator of proximity between two switching windows. A high overlap ratio between nodes u, v predicts that the nodes are closely coupled with each other and change in the switching window of one node will surely lead to the change in switching window of other node. The intuition governing this observation is that if two switching windows are closely coupled with each other with MCF as 1 (considering intrinsic coupling capacitance without switching effects), they are most likely to remain closely coupled when MCF is varied due to switching effects. Overlap ratio defines the closeness of the coupling between two nodes based on the current windows. Also if the overlap ratio between the nodes u, v of some coupling edge is high, it becomes a potential candidate for local coupling edges over some other coupling edge whose nodes w, x do not have a high overlap ratio.

D. Coupling Partitioning

The set $C - C_P^G$ are now left with edges as shown in Section IV-B. The coupling edge (e, f) has an important property. Timing information on e can be affected only because of the coupling with f. We characterize such coupling edges as *Local*. *Local Coupling Edges* C^L is a subset of edges C such that $\forall(u, v) \in C^L$ any change in rise and fall delay window at node u does not propagate to node v through the fan-outs of node u. In other words, only coupling edge between u, v can change the windows on the respective gates. If we take a look at edges (a, b) and (c, d) from Fig. 20, both the edges can be potential candidate for *Local Coupling Edges*. If we arbitrarily select any one of them as *Local*, it will force the participating nodes of other coupling edge related by the fan-out relation. This is because once we choose one edge as *Local* we must update them simultaneously. It in-turn forces the two nodes to be considered as one merged node in the graph G.

Formation of such a merged node change the fan-out relation of the original graph. If we choose (a, b) as local coupling edge, we need to look at them as a single node $a - b$, and therefore it will force nodes c and d to be related by fan-out relation. We call (c, d) global coupling only if (a, b) is considered as local. *Global Coupling Edges* C^G is a subset of edges C such that $\forall(u, v) \in C^G$ vertex u is related to vertex v by fan-in relation, i.e., there exists a path from u to v in G^F or $\hat{G^F}$. $\hat{G^F}$ is obtained by a transformation on graph G^F. The transformation forms a merged node in the original graph by choosing some couplings as local, and thus changes the fan-in relation of the whole graph.

Problem 2: Coupling Partitioning is defined as to identify feedback edges $e \in C - C_P^G$ such that the sum of coupling weights on those edges is minimized.

Due to interdependence of edges in $C - C_P^G$, identifying minimal coupling weighted feedback edges is not straightforward. In fact, we can give the following theorem

Theorem 3: Identification of feedback edges $e \in C - C_P^G$ such that the sum of coupling weights on the edges is minimized is NP-complete.

We present a proof-sketch for NP-completeness. Choosing an edge e from $C - C_P^G$ and considering it as a local coupling edge will make several other coupling edges as non-primary global edges. Thus we can have a conflict graph between edge e and the non-primary global edges it made. Minimizing the coupling weight can be reduced to finding a minimal independent set in the conflict graph, which is a NP-complete problem.

Algorithm: Identify-NonPrimary-Global-Coupling

```
Input:Coupling Edge (u,v), G^F, G^C
Output:NonPrimary Global Coupling(NPGC)
begin
  NPGC = ∅
  Generate Ḡ^F: Complement of G^F
  Collect Ancestor(u) from Ḡ^F
  Collect Descendent(v) from G^F
  while(∃e ∈ C : e = (i,j), i ∈ Ancestor(u),
          j ∈ Descendent(v))
    NPGC←NPGC ∪ e
  Collect Descendent(u) from G^F
  Collect Ancestor(v) from Ḡ^F
  while(∃e ∈ C : e = (i,j), i ∈ Descendent(u),
          j ∈ Ancestor(v)
    NPGC←NPGC ∪ e
end
```

Fig. 21. Identify non–primary global coupling.

To solve the problem for general circuits we use the following heuristic. We sort the coupling edges $C - C_P^G$ by decreasing overlap-ratio. We select the edge with maximum overlap-ratio and insert the edge into C^L. Now as discussed above choosing the edge as a member of local coupling will change the structure of the circuit and might generate edges for C_{NP}^G, where C_{NP}^G is the set of non primary global edges. Non-primary global edges are basically those feedback edges on which we are minimizing the sum of coupling weights. We identify such edges by the algorithm presented in Fig. 21. Following that we remove the selected edge and the edges generated by the algorithm from coupling edges. We choose the next maximum overlap-ratio from remaining coupling edges and repeat the procedure until all the edges of $C - C_P^G$ are considered. The algorithm takes as input the coupling edge e which denotes the coupling between nodes u and v. We define relations *Ancestor* and *Descendent*.

Definition 9: Ancestor of a node u is the set of nodes, such that there exist a simple path between each node of the set to the node u traversing the fan-in edges of the graph G.

Definition 10: Descendent of a node u is the set of nodes, such that there exist a simple path from node u to each node of the set traversing the fan-in edges of the graph G.

As a designer's point of view, Ancestor and Descendent set of a node u are fan-in and fan-out cone of the u.

The algorithm shown in Fig. 21. On each node $v \in G^F$ list of all the edges which fans out from node v is kept. By Definition 10, G^F is ideal to find the descendents of a given node v by traversing the graph identifying which all nodes can be reached through the fan-out edges from node v. Representation as G^F is not ideal for getting ancestors (by Definition 9). Therefore from G^F we generate the complement $\overline{G^F}$. $\overline{G^F}$ is obtained by reversing the direction of each edge of G^F and thus keeping the information on each node $v \in \overline{G^F}$ list of all edges which fan-in to node v. Beginning a traversal at node v all ancestors of the

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

node can be obtained by the list at each node in the complement graph $\overline{G^F}$.

Using the edge list information at each nodes of G^F and $\overline{G^F}$, generating the sets Ancestor and Descendent are straightforward. Ancestor(u) and Descendent(u), respectively, denotes the sets of Ancestor and Descendent corresponding to the node u. The collection of nodes is done by a BFS-based routine [13] that looks at the top of the queue element, pops it up and stores it in another list called Ancestor or Descendent depending upon the graph on which the routine is called. We are basically seeking for the elements of NPGC which is initially set as $\emptyset$. If there is n coupling edge (i, j), where i belongs to the Ancestor set of u and j belongs to the Descendent set of v, then merging the nodes i, j into $i-j$ will make the coupling edge (i, j) a member of non-primary global coupling. We identify all such edges in the *while* loop of the algorithm and store them in the set NPGC. Elements of set NPGC is added to C^G_{NP} and following that we remove the appropriate elements from E^C as discussed before. C^G is crucial for iteration algorithm as any change in the timing window of one node related to the global coupling edge will eventually propagate to the other node and we can prune iterations based on this fact.

E. Iteration Algorithm

The algorithm we present in Fig. 22 is based on the theoretical foundation established in [8]. We call this algorithm *Fast chaotic iterator* since the iterative process the algorithm uses is known as *chaotic iteration* [9]. Our algorithm decides the timing propagation order based on coupling edge partitioning. First of all a timing iteration considering intrinsic coupling capacitance on all nodes is done. Identification of global and local couplings in Section IV-D, is used to change the order in which the nodes are added to the queue. Global couplings are all those feedback edges, whose deletion will make the general graph G a DAG if the local couplings are considered as a single updating unit. We can process the graph in a topological order once the feedback edges are broken and we need to start the timing iteration again only if the windows at the feedback edges change.

We introduce two hypothetical nodes PO and PI. In the generated timing graph, PI is connected to inputs of the circuit while all outputs are connected to PO. All the nodes are sorted by topological order. We start the iterative scheme by getting the topmost node of the queue. Now we compute the coupling factors considering that particular node as victim and the rest as aggressors. Detailed description of coupling factor computation is given in Section II. Accordingly, we update the effective capacitance on the victim node which comes from the sum of the gate, interconnect, and coupling capacitance. Based on accuracy requirements there are two methods to generate the effective coupling capacitance; Multiplying coupling capacitance with coupling factors computed by method shown in Section II-B or using converged best/worst fixed point capacitances from directed search technique presented in Section II-C. If the timing windows on the victim node are changed, then we need to process all its local aggressors. Given vertices u, v, $u \rightarrow v$ means that a path exists from vertex u to v in graph G.

```
Algorithm: Fast-Chaotic-Iterator
Input:General Graph G
Output:Timing Windows with Coupling
     Capacitance Effect
begin
Procedure:Initialization
 Node-Queue← ∅
 while(∃u ∈ V : u ∉ Node-Queue)
   UpdateEffectiveCapacitance(u,1)
   ComputeWindows(u)
   Node-Queue←Node-Queue∪u
Procedure:Modified Chaotic Iterations
 while(Node-Queue≠ ∅)
   u = Pop(Node-Queue)
   while(∃v ∈ C : u → v)
     k^u ← ComputeCouplingFactors(u,v)
     UpdateEffectiveCapacitance(u,k^u)
   ComputeWindows(u)
   if(Timing Windows on u Changes)
     while(∃v ∈ F : u → v)
       Insert v to Node-Queue uniquely
     while(∃v ∈ C : u → v)
       if(v ∉ C^G)
         Insert v to Node-Queue uniquely
end
```

Fig. 22. Fast chaotic iterator.

Here we want to mention the changes needed to our proposed iteration algorithm to use an accurate model shown in Section III. For each victim aggressor pair, our current-based model computes a crosstalk induced delta delay. Given victim and aggressor u and v, let the procedure which computes crosstalk induced speedup/slowdown be $\mathrm{ComputeDeltaDelay}(u, v)$. For a victim cluster we superimpose the crosstalk induced delta delay following maximum realizable aggressor set (MRAS) formulation from [25]. Now in the algorithm shown in Fig. 22, replacing the procedure $\mathrm{ComputeCouplingFactors}(u, v)$ by $\mathrm{ComputeDeltaDelay}(u, v)$ is enough to replace the heuristic model with the SPICE accurate current source based coupling delay model.

Coupling factor based models are widely used in industry for early design stages because of its efficiency. In a preliminary version of this paper [17], we did experiments with directed search technique Section II-C. Our experiments validate that a coupling factor based model is efficient for iterative timing analysis while sacrificing not much on accuracy. For the purpose of illustration in this paper we used heuristic coupling model from Section II-B. In a coupling factor-based timing analysis algorithm, $\mathrm{ComputeWindows}$ function is the most costly operation as it computes the new windows from the input windows at the fan-in edges to the victim node. $\mathrm{ComputeCouplingFactor}$ takes a victim and aggressor node, and on the basis of their rise delay, fall delay, rise slew, and fall slew windows, generates the effective coupling factors. To compare the performance of our algorithm, we developed a general iterative algorithm which does not consider coupling partitioning.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

TABLE I
MODEL DEVIATION RESULTS

Circuit	# of Nodes	# of CE	012 Model [2]			Proposed Model		
			RT(s)	TA	(rd_l, rd_h)	RT(s)	TA	(rd_l, rd_h)
c432	198	277	0.05	1197	(0.02,24.43)	0.13	3004	(0.32,31.69)
c499	245	312	0.07	1206	(0.07,26.93)	0.28	4530	(0.02,19.15)
c880	445	620	0.22	4440	(0.16,26.98)	0.45	11595	(0.03,37.62)
c1355	589	829	0.23	6470	(0.19,31.01)	0.96	24291	(0.56,35,57)
c1908	915	1332	0.46	13009	(0.20,49.40)	1.75	45716	(0.22,53.93)
c2670	1428	1929	0.36	10094	(0.01,55.58)	1.35	34165	(0.01,50.41)
c3540	1721	2545	0.45	12822	(0.14,78.20)	2.28	60010	(0.32,71.17)
c5315	2487	3542	0.68	19467	(0.03,53.12)	3.74	67904	(0.01,60.08)
c6288	2450	3622	2.74	47135	(0.15,158.77)	15.48	246879	(0.38,167.67)
c7552	3721	5412	1.38	36887	(0.01,55.88)	6.71	127572	(0.01,64.67)

TABLE II
PERFORMANCE ENHANCEMENT RESULTS

Circuit	# of CE	CI		Fast-CI				
		TA	(rd_l, rd_h)	Global	TA	(rd_l, rd_h)	%Speedup	P-RT (s)
c432	277	3004	(0.32,31.69)	195	2831	(0.32,31.16)	5.7	0.04
c499	312	4530	(0.02,19.15)	176	3437	(0.02,19.64)	24.1	0.08
c880	620	11595	(0.03,37.62)	243	7289	(0.03,37.18)	37.1	0.13
c1355	829	24291	(0.56,35,57)	532	9207	(0.52,35.08)	62.1	0.19
c1908	1332	45716	(0.22,53.93)	679	27856	(0.20,53.30)	39.1	0.49
c2670	1929	34165	(0.01,50.41)	550	31474	(0.01,49.95)	7.8	1.21
c3540	2545	60010	(0.32,71.17)	973	44690	(0.32,69.33)	25.5	2.39
c6288	3622	246879	(0.38,167.67)	2503	163877	(0.21,165.48)	33.6	7.19
c7552	5412	127572	(0.01,64.67)	1388	119166	(0.01,63.14)	6.6	6.09

V. EXPERIMENTAL SETUP

A. Circuit Modeling

We model a given circuit as a directed acyclic graph (referred to subsequently as the circuit's *timing graph*). Nodes in the timing graph represent gates in the circuit while the edges represent the corresponding interconnects. We map all nodes to logic gates from the Faraday 90-nm technology library. Delay models are available from lookup tables that yield a gate's delay and slew as functions of its input slew and load (output capacitance). Extracted coupling capacitance values are used to generate a *coupling graph* that denotes the timing dependencies introduced due to coupled nets. This coupling graph is then superimposed on the timing graph. We consider that each net is coupled to at most four other nets. However, this is not a limitation in our approach.

B. Results

We present three sets of results. Table I shows the deviations of the converged timing windows from our coupling model over a discrete model based on MCFs 0,1,2 [2] on ISCAS85 benchmarks [14]. CE denotes number of coupling edges, RT shows runtime in seconds and TA is the number of gate-delay lookup table accesses. (rd_l, rd_h) shows the rise delay window at PO. Since in our coupling model the best coupling factor can vary between -1 and 1 while the worst can vary between 1 and 3, the final windows obtained by MCFs 0,1,2 are conservative in some cases whereas non-conservative in other cases. Therefore as physically coupling capacitance can indeed be as large as $3 \times C_c$ and as small as $-1 \times C_c$, designing with coupling factors 0,1,2 can lead to large errors. We used an iterative algorithm [6] and used the two coupling models to obtain the results.

Performance enhancement results of our efficient iteration scheme is shown in Table II. For these experiments, we used the proposed coupling model but changed the solver algorithm. CI denotes the chaotic iterator (iterative algorithm based on [6]) and we represent our algorithm by $Fast{-}CI$. Number of global edges obtained by our algorithm is shown as *Global*. $P-RT$ shows the run-time of coupling edge partitioning. To demonstrate the speed-up obtained from our approach, we define the following:

$$\%Speedup = \frac{TA^{FastCI} - TA^{CI}}{TA^{CI}} \times 100.0.$$

$\%Speedup$ denotes the percentage of gate-delay table-looks saved using our approach in comparison to CI. We choose this metric since the table-lookup operations are run-time expensive operations in timing analysis. From the table, we observe that our approach reduces table accesses on an average by 26.8%, and by 62.1% in the best case. The error in timing windows are negligible. We also made an interesting observation for circuit c5315. We didn't get any speedup over the iterative approach. The reason for the same is attributed to the number of global coupling edges being much less than the number of local coupling edges.

Finally, we simulated our algorithm using a complex coupling model as shown in Section III. Results are presented in Table III. Second column shows the ratio of global coupling edges and total coupling edges in the circuit. Third and fourth columns respectively show the runtime of traditional algorithm CI and

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

TABLE III
RUNTIME COMPARISON

Circuit	Ratio	CI(s)	Fast-CI(s)	Speed-up
c432	0.70	90.83	84.35	1.1
c499	0.56	129.04	96.15	1.34
c880	0.39	343.14	214.77	1.60
c1355	0.64	684.76	253.89	2.70
c1908	0.51	1282.33	772.64	1.66
c2670	0.29	1038.22	938.89	1.1
c3540	0.38	1704.11	1246.31	1.38
c6288	0.69	6798.83	4562.93	1.50
c7552	0.26	3706.74	3558.05	1.04

our algorithm $Fast - CI$. It is quite interesting to see the correlation between number of global edges and the improvement in runtimes. For circuit likes c1908 and c6288 our algorithm is 1.66× and 1.5× faster than the traditional algorithm. Due to widespread use of coupling aware timing analysis algorithm in VLSI design flow, such speedups are crutial. In fact the runtimes for coupling edge partitioning (P-RT from Table II) is insignificant compared to the timing analysis runtimes. Therefore a run of coupling partitioning to determine if the circuit is dominated by global coupling edges is essential. If we have a favorable coupling structure (Simiar to circuit c1355) our algorithm can achieve 2.7× speedup compared to traditional approach. Results shown are for experiments performed on a Xeon 2.4 GHz processor server having 1 Gb RAM and running RedHat Linux 9.0.

VI. CONCLUSION AND FUTURE WORK

We present a framework for fast and accurate static timing analysis considering coupling. Two variants of waveform based coupling delay model is presented and a technique to speed up iterations in timing analysis is proposed. We present SPICE comparisons to demonstrate the accuracy of our proposed current based coupling model. We present experimental results on the ISCAS benchmarks to demonstate the efficiency of our iteration algorithm under different coupling models. We also demonstrate speedups of up to 62.1%, and average speedups by 26.8% using our efficient iteration scheme with negligible error in timing windows. With a complex coupling model we achieved maximum speedup of 2.7× and average speedup of 1.5×. It is thus evident that our framework improves both the accuracy and the efficiency of a coupling aware static timer significantly. In the future, we will extend our current based coupling model to consider both capacitive and inductive coupling.

ACKNOWLEDGMENT

The first author, D. Das, would like to thank Dr. W. Scott, Magma Design Automation for providing a research version of current source-based coupling model. He would also like to thank Faraday Corporation for providing their experimental 90-nm ASIC cell libraries.

REFERENCES

[1] R. Kumar, "Interconnect and noise immunity design for the Pentium 4 processor," in *Proc. Des. Autom. Conf.*, 2003, pp. 938–943.
[2] S. S. Sapatnekar, "A timing model incorporating the effect of crosstalk on delay and its application to optimal channel routing," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 19, no. 5, pp. 550–559, May 2000.
[3] A. Kahng, S. Muddu, and E. Sarto, "On switch factor based analysis of coupled RC interconnects," in *Proc. Des. Autom. Conf.*, 2000, pp. 79–84.
[4] P. Chen, D. A. Kirkpatrick, and K. Keutzer, "Miller factor for gate-level coupling delay calculation," in *Proc. Int. Conf. Comput.-Aided Des.*, San Jose, CA, Nov. 2000.
[5] M. Ghoneima and Y. Ismail, "Accurate decoupling of coupled on-chip buses," in *Proc. Int. Symp. Circuits Syst.*, 2005, pp. 4146–4149.
[6] P. Chen, D. A. Kirkpatrick, and K. Keutzer, "Switching window computation for static timing analysis in presence of crosstalk noise," in *Proc. Int. Conf. Comput.-Aided Des.*, San Jose, CA, Nov. 2000.
[7] R. Arunachalam, K. Rajagopal, and L. T. Pilleggi, "Taco: Timing analysis with coupling," in *Proc. Des. Autom. Conf.*, Los Angeles, CA, Jun. 2000, pp. 266–269.
[8] H. Zhou, "Timing analysis with crosstalk is a fixpoint on a complete lattice," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 22, no. 9, pp. 1261–1269, Sep. 2003.
[9] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proc. ACM Symp. Principles Program. Lang.*, Los Angeles, CA, Jan. 1977, pp. 238–252.
[10] F. Dartu and L. T. Pileggi, "Calculating worst-case gate delays due to dominant capacitance coupling," in *Proc. Des. Autom. Conf.*, Anaheim, CA, Jun. 1997, pp. 46–51.
[11] P. D. Gross, R. Arunachalam, K. Rajagopal, and L. T. Pileggi, "Determination of worst-case aggressor alignment for delay calculation," in *Proc. Int. Conf. Comput.-Aided Des.*, San Jose, CA, Nov. 1998, pp. 212–219.
[12] M. R. Garey and D. S. Johnson, *Computers and Intractability*. Anaheim, CA: Freeman and Co., 1979.
[13] T. H. Cormen, C. E. Leiserson, and R. H. Rivest, *Introduction to Algorithms*. Cambridge, MA: MIT Press, 1989.
[14] F. Brglez and H. Fujiwara, "A neutral netlist of 10 combinatorial benchmark circuits," in *Proc. Int. Symp. Circuits Syst.*, 1985, pp. 695–698.
[15] T. Sakurai and A. R. Newton, "Alpha-power law MOSFET model and its applications to CMOS inverter delay and other formulas," *IEEE J. Solid-State Circuits*, vol. 25, no. 2, pp. 584–594, Apr. 1990.
[16] B. A. Davey and H. A. Priestley, *Introduction to Lattices and Order*. Cambridge, MA: Cambridge University, 1990.
[17] D. Das, A. Shebaita, Y. Ismail, H. Zhou, and K. Killpack, "Nostra-XTalk: A predictive framework for accurate static timing analysis in UDSM VLSI circuits," in *Proc. ACM Great Lakes Symp. VLSI*, 2007, pp. 25–30.
[18] D. Das, W. Scott, S. Nazarian, and H. Zhou, "An efficient current-based logic cell model for crosstalk delay analysis," in *Proc. IEEE ISQED*, 2009, pp. 627–633.
[19] J. Qian, S. Pullela, and L. T. Pileggi, "Modeling the effective capacitance of the RC interconnect," *IEEE Trans. Comput.-Aided Des. Integr. Circuits*, vol. 13, no. 12, pp. 1526–1535, Dec. 1994.
[20] J. F. Croix and D. F. Wong, "Blade and razor: Cell and interconnect delay analysis using current-based models," in *Proc. DAC*, 2003, pp. 386–389.
[21] I. Keller, K. Tseng, and N. K. Verghese, "A robust cell-level crosstalk delay change analysis," in *Proc. ICCAD*, 2004, pp. 147–154.
[22] H. Fatemi, S. Nazarian, and M. Pedram, "Statistical logic cell delay analysis using a current-based model," in *Proc. DAC*, 2006, pp. 253–256.
[23] C. V. Kashyap, C. S. Amin, N. Menezes, and E. Chiprout, "A nonlinear cell macromodel for digital applications," in *Proc. ICCAD*, 2007, pp. 678–685.
[24] F. Dartu and L. T. Pileggi, "Calculating worst-case gate delays due to dominant capacitance coupling," in *Proc. DAC*, Anaheim, CA, Jun. 1997, pp. 46–51.
[25] A. Glebov, S. Gavrilov, D. Blaauw, S. Sirichotiyakul, C. Oh, and V. Zolotov, "False noise analysis using logic implications," in *Proc. ICCAD*, 2001, pp. 515–521.
[26] D. Das, A. Shebaita, H. Zhou, Y. Ismail, and K. Killpack, "FA-STAC: A framework for fast and accurate static timing analysis with coupling," presented at the ICCD, San Jose, CA, 2006.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.

Debasish Das (S'06) received the B.Tech. (honors) degree in computer science and engineering from Indian Institute of Technology, Kharagpur, India, in 2004. He is currently pursuing the Ph.D. degree in electrical engineering and computer science from Northwestern University, Evanston, IL.

His research interests include mathematical modeling, algorithm design and computer-aided design for VLSI circuits, especially static timing analysis and optimization.

Ahmed Shebaita received the B.S. and M.S. degrees in electrical engineering from Cairo University, Cairo, Egypt, in 1999 and 2003, respectively, and the Ph.D. degree in electrical engineering and computer science from Northwestern University, Evanston, IL, in 2009.

He is currently an IT Architect with IBM, Valencia, Spain. His research interests include digital integrated circuit design, interconnect analysis, circuit modeling and circuit simulation with applications to static timing analysis.

Hai Zhou (SM'04) received the B.S. and M.S. degrees in computer science and technology from Tsinghua University, Beijing, China, in 1992 and 1994, respectively, and the Ph.D. degree in computer science from the University of Texas, Austin, in 1999.

He is an Associate Professor with the Departmen tof Electrical Engineering and Computer Science, Northwestern University, Evanston, IL. His research interests include VLSI computer-aided design, algorithm design, and formal methods.

Dr. Zhou was a recipient of a CAREER Award from the National Science Foundation in 2003.

Yehea Ismail (SM'09) was born in Giza, Egypt, on November 16, 1971. He received the B.Sc. degree in electronics and communications engineering with distinction and honors and the Masters degree in electronics from Cairo University, Cairo, Egypt, in 1993 and 1996, respectively, and the Masters degree in electrical engineering and Ph.D. degree from the University of Rochester, Rochester, NY, in 1998 and 2000, respectively.

As one of the top of his class, he was appointed as a Teacher Assistant with the Department of Electrical and Computer Engineering, Cairo University, in August 1993. He is currently an Associate Professor with Northwestern and Nile University and Director of the Nanoelectronics Integrated Systems Center. He has coauthored more than 130 technical papers and a book, and several book chapters. He was with IBM Microelectronics from 1997–1999. His primary research interests include digital integrated circuit design, interconnect, noise, innovative circuit simulation, and related circuit level issues in high performance VLSI circuits.

Dr. Ismail is the chair elect of the CAS VLSI Technical Committee, Associate Editor-in-Chief of the IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS, was on the Editorial Board of the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—PART I: FUNDAMENTAL THEORY AND APPLICATIONS, and a Guest Editor for a special issue of the IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS on On-Chip Inductance in High Speed Integrated Circuits. He has also chaired many conferences such as GLSVLSI, IWSOC, ISCAS. He was selected as the 2002 IEEE Circuits and Systems Society Outstanding Young Author Award winner, and received the National Science Foundation Career Award in 2002. He was a recipient of the Best Teacher Award in the ECE Department, Northwestern University, in 2002–2003.

Kip Killpack received the B.S. and M.S. degrees in electrical and computer engineering from the University of Utah, Salt Lake City.

He is currently a Design Engineer and Silicon Debugger with Intel Corporation, Hillsboro, OR. Prior to this, he was a CAD Researcher with Strategic CAD Laboratories, Intel. His research interests include presilicon timing-analysis accuracy, postsilicon timing correlation, RTL coding, schematic and layout design, silicon debug, and electrical-effects modeling.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:05:55 UTC from IEEE Xplore. Restrictions apply.